

PS03 - Overfitting

Zachary Tipton

2026-02-10

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.1.4      v readr      2.1.5
v forcats    1.0.1      v stringr    1.5.2
v ggplot2     4.0.0      v tibble     3.3.0
v lubridate  1.9.4      v tidyr      1.3.1
v purrr       1.1.0
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(fixest)
```

Model Specification and Prediction Error

In this problem set, we'll explore how different model specifications perform when the true functional form is unknown and non-linear. We'll generate data from a complex data generating process and evaluate how well different regression models capture the underlying relationship.

```
source(
  "https://gist.githubusercontent.com/kylebutts/06908e23880457e2d7fb69444f7c21c0/raw/8900325
)
```

The above code cell loads our secret data-generating function, which you can call with `draw_sample(N = 500)`. It returns a dataset with four variables: - The outcome `y` - Two continuous covariates, `x1` and `x2` - A discrete variable called `group`. (Group A, B, or C)

The outcome is generated through a non-linear combination of these covariates

Model Evaluation Function

We'll create a function that generates data, fits a regression model with a specified formula, and returns the model fit measured by Mean Squared Prediction Error (MSPE).

```
## Function to evaluate model specification
evaluate_model <- function(
  formula,
  N = 500, # training data
  B = 100, # repeat this many times
  in_sample = TRUE, # can toggle to FALSE for oos mspe
  seed = NULL # would normally set to get the same numbers each output
) {
  if (!is.null(seed)) {
    set.seed(seed)
  }

  mspes <- vector("numeric", B)
  for (b in 1:B) {
    ## Generate training data
    train_data <- draw_sample(N = N)

    ## Fit model on training data
    model <- feols(formula, data = train_data) # fit the model, pass training data

    if (in_sample) {
      ## In-sample MSPE: predict on training data
      predictions <- fitted(model)
      actuals <- train_data$y

      ## Calculate in-sample MSPE
      mspe <- mean((actuals - predictions)^2)
    } else {
      ## Generate fresh test data (different observations, same size)
      test_data <- draw_sample(N = N)

      ## Predict on test data
      predictions <- predict(model, newdata = test_data)
      actuals <- test_data$y
    }
  }
  mspes
}
```

```

    ## Calculate out-of-sample MSPE
    mspe <- mean((actuals - predictions)^2)
  }

  mspes[b] <- mspe
}

return(mspes)
}

```

Example 0: Intercept only

To get a sense of how a truly bad model would perform, let's use just an intercept as our formula: - totally useless model

```

baseline_mspe <- evaluate_model(
  formula = y ~ 1,
  N = 500,
  in_sample = TRUE
)

mean(baseline_mspe)

```

```
[1] 221.4271
```

Example 1: Simple Linear Model

Let's start with a simple linear specification and evaluate its performance. - does a lot better than example 0, but we don't know how well.

```

simple_linear_mspe <- evaluate_model(
  formula = y ~ x1 + x2 + i(group), # two continuous variables and indicator variables for group
  N = 500,
  in_sample = TRUE
)

mean(simple_linear_mspe)

```

```
[1] 13.95152
```

The simple linear model provides a baseline. However, since the true relationship is non-linear, we expect this specification to miss important patterns in the data.

Example 2: Adding Polynomial Terms

Let's improve our model by adding quadratic terms to capture curvature in the relationship.

```
poly_mspe <- evaluate_model(  
  formula = y ~ x1 + I(x1^2) + x2 + I(x2^2) + I(x1 * x2) + i(group), # I to do math, i for interaction  
  N = 500,  
  in_sample = TRUE  
)  
  
mean(poly_mspe)
```

```
[1] 5.515885
```

By adding polynomial terms, we allow the model to capture non-linear relationships between the continuous covariates and the outcome. Compare the MSPE to the simple linear model.

In-Class Challenge: Find the Best Model

Challenge: Using the `evaluate_model` function, find the regression model with the **lowest MSPE**. The student with the best model will win a prize!

Rules: - You can use any combination of `x1`, `x2`, and `i(group)` - You can include polynomial terms (e.g., `I(x1^2)`, `I(x1^3)`) - You can include interaction terms (e.g., `x1 * x2`, `i(group, x1)`) - Use `N = 500` and `set.seed(20260210)` for consistency - The goal is to minimize the MSPE

Tips: - Try different combinations of main effects - Experiment with polynomial degrees - Test various interaction structures - Consider how `group` might modify the effects of `x1` and `x2`

Your Best Model

Once you've found your best model, report it below. Make sure to use `set.seed(20260210)` and `N = 500`! Use the tips below for adding different kinds of terms in your model

```
## Experiment with different model specifications here
## Try different combinations of variables, polynomials, and interactions
my_best_formula <- y ~ poly(x1, 3) + poly(x2, 3) + I(x1 * x2) + i(group) ## Change this!

# actual formula <- y ~ x1 + x2 + x2^2 + x1^2 + I(x1 * x2) + i(group, x1) + i(group, x2) + i(

my_best_mspe <- evaluate_model(
  formula = my_best_formula,
  N = 500,
  seed = 20260210
)

cat("My best in-sample MSPE:", mean(my_best_mspe), "\n")
```

My best in-sample MSPE: 5.501862

```
cat("Formula:", deparse(my_best_formula), "\n")
```

Formula: y ~ poly(x1, 3) + poly(x2, 3) + I(x1 * x2) + i(group)

Tips: Flexible Model Specification with fixest

The `fixest` package provides powerful tools for flexible model specification. Here are some useful techniques:

Polynomial Terms

Use `I()` to include polynomial terms:

```
## Quadratic specification
formula = y ~ x1 + I(x1^2) + x2 + I(x2^2) + i(group)

## Cubic specification
formula = y ~ x1 + I(x1^2) + I(x1^3) + x2 + i(group)
```

Interactions Between Continuous Variables

Use `:` to interact two continuous variables without including the main effects automatically:

```
## Interaction only (no main effects)
formula = y ~ x1:x2 + i(group)

## Interaction with main effects (equivalent to x1 * x2)
formula = y ~ x1 + x2 + x1:x2 + i(group)
```

Categorical Indicators

Use `i()` to create indicator variables for categorical covariates:

```
## Create indicators for group categories (omits first level by default)
formula = y ~ x1 + x2 + i(group)

## Same as: y ~ x1 + x2 + i(group) (since group is already categorical)
```

Varying Slopes (Interactions with Categories)

Use `i(cat, var)` to allow the slope of a continuous variable to vary by category:

```
## Allow x1 slope to differ by group category
formula = y ~ x1 + x2 + i(group, x1)

## This creates: x1 + x2 + i(group) + x1:group_B + x1:group_C
## (slope of x1 differs for each level of group)

## Combine with varying intercepts and slopes
formula = y ~ x2 + i(group) + i(group, x1)
## Creates: x2 + i(group) + x1:group_B + x1:group_C
```

Out-of-Sample Performance

Now, let's see how well your model actually performs. Evaluate your **best model** (the one you submitted above) on fresh testing data by setting `in_sample = FALSE`.

This will train your model on one set of 500 observations and evaluate it on a completely different set of 500 observations.

```
## Evaluate YOUR best model on out-of-sample data
my_best_in_sample <- evaluate_model(
  formula = my_best_formula,
  N = 200,
  in_sample = TRUE,
  seed = 20260204
)

my_best_out_sample <- evaluate_model(
  formula = my_best_formula,
  N = 200,
  in_sample = FALSE,
  seed = 20260204
)

cat(
  "My best model in-sample MSPE:",
  round(mean(my_best_in_sample), 3),
  "\n"
)
```

My best model in-sample MSPE: 5.218

```
cat(
  "My best model out-of-sample MSPE:",
  round(mean(my_best_out_sample), 3),
  "\n"
)
```

My best model out-of-sample MSPE: 6.103

How did your model perform? Did the out-of-sample MSPE surprise you?

- I originally went about cranking the hell out of the polynomials. My in-sample was starting to look quite nice. And then my OOS ended up being like 100,000+ haha. Perfect overfitting example. So I then started to back off. I did not get in the ~4.xx like some of the better predictions, but 5.2-ish is still respectable.

Exercise

1. Which model won the in-sample competition? How complicated was the winning formula? Austin of course won. He's way smarter than us econ nerds. His model had all the right interactions. The right grit. The right determination. It was the right model at the right time. As I'm pushed further down the rung by the next generation I acknowledge that I only have myself to blame.
2. How did the winning model perform out-of-sample? Did it generalize well, or did it overfit?

Generalized well.

Understanding Overfitting

Bias-Variance Tradeoff at Fixed Sample Size

So far, we've compared specific models. Now let's systematically explore how model complexity affects prediction error for a fixed sample size. As we increase model complexity (adding higher-order polynomials and more interactions), we expect to see:

- **In-sample MSPE:** Should continuously decrease or stay flat (more flexible models can always fit training data at least as well)
- **Out-of-sample MSPE:** Should initially decrease (reducing bias) but eventually increase (overfitting, increasing variance)

This U-shaped pattern for out-of-sample error illustrates the **bias-variance tradeoff**.

```
## Function to evaluate models with increasing polynomial complexity
evaluate_complexity <- function(
  N = 500,
  max_degree = 6,
  B = 100,
  seed = NULL
) {
  if (!is.null(seed)) {
    set.seed(seed)
  }
}
```



```

}

results <- data.frame(
  degree = integer(),
  model_type = character(),
  mspe_type = character(),
  mspe = numeric()
)

for (degree in 1:max_degree) {
  ## Build formula with polynomial terms up to current degree
  ## Include interactions between x1 and x2 polynomials
  poly_terms_x1 <- paste0("I(x1^", 1:degree, ")", collapse = " + ")
  poly_terms_x2 <- paste0("I(x2^", 1:degree, ")", collapse = " + ")

  ## Add interactions between all polynomial terms
  if (degree >= 2) {
    interaction_terms <- c()
    for (i in 1:degree) {
      for (j in 1:degree) {
        if (i + j <= degree + 1) {
          ## Cap interaction order
          interaction_terms <- c(
            interaction_terms,
            paste0("I(x1^", i, ")", ":", "I(x2^", j, ")")
          )
        }
      }
    }
    interaction_str <- paste(interaction_terms, collapse = " + ")
    formula_str <- paste(
      "y ~",
      poly_terms_x1,
      "+",
      poly_terms_x2,
      "+",
      interaction_str,
      "+ i(group)"
    )
  } else {
    formula_str <- paste(
      "y ~",

```

```

        poly_terms_x1,
        "+",
        poly_terms_x2,
        "+ i(group)"
    )
}

formula <- as.formula(formula_str)

## Evaluate in-sample
mspe_in <- evaluate_model(formula, N = N, B = B, in_sample = TRUE)

## Evaluate out-of-sample
mspe_out <- evaluate_model(formula, N = N, B = B, in_sample = FALSE)

## Count parameters (approximate)
n_params <- degree * 2 ## x1 and x2 polynomials
if (degree >= 2) {
    n_params <- n_params + length(interaction_terms)
}
n_params <- n_params + 2 ## group indicators

results <- rbind(
    results,
    data.frame(
        degree = rep(degree, 2),
        n_params = rep(n_params, 2),
        mspe_type = c("In-sample", "Out-of-sample"),
        mspe = c(mean(mspe_in), mean(mspe_out))
    )
)
}

return(results)
}

## Run the complexity evaluation
N = 100
complexity_results <- evaluate_complexity(
    N = N,
    max_degree = 6,
    B = 50,

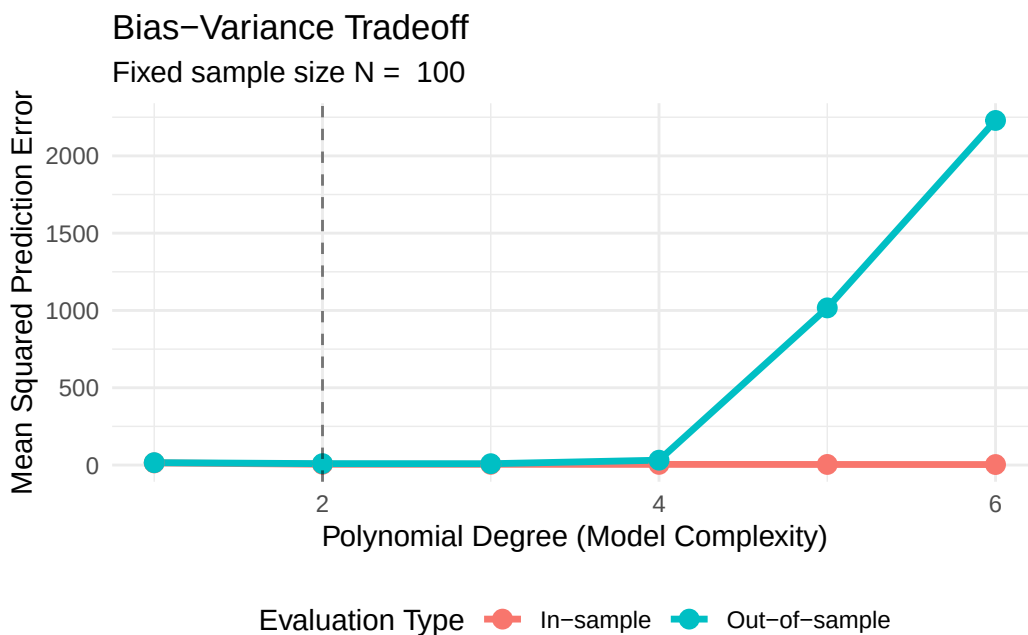
```

```

seed = 20260210
)

## Plot the bias-variance tradeoff
ggplot(complexity_results, aes(x = degree, y = mspe, color = mspe_type)) +
  geom_line(linewidth = 1.2) +
  geom_point(size = 3) +
  geom_vline(xintercept = 2, linetype = "dashed", alpha = 0.5) +
  labs(
    title = "Bias-Variance Tradeoff",
    subtitle = paste("Fixed sample size N = ", N),
    x = "Polynomial Degree (Model Complexity)",
    y = "Mean Squared Prediction Error",
    color = "Evaluation Type"
  ) +
  theme_minimal() +
  theme(legend.position = "bottom")

```



Exercise

1. Interpreting the Bias-Variance Tradeoff Plot:

- At what degree does the out-of-sample MSPE reach its minimum? A: 2

- Why does the in-sample MSPE continue to decrease (or flatten) as complexity increases?
A: adding more terms can only reduce in-sample MSPE mechanically. As more terms are added a little more noise is explained, and a little more, and a little more, and a little more, and a little more, and a little more, and a little more
 - What happens to the gap between in-sample and out-of-sample MSPE as model complexity increases? A: if we get too complicated with the model, the in-sample MSPE looks great, but the OOS MSPE goes to the moon, greatly increasing the gap. This happens because the model is overfit and individual observations are pulling the model around.
2. **Optimal Complexity:** The true data generating process uses degree-2 polynomials for x_1 and x_2 . Does the minimum out-of-sample MSPE occur at degree 2? If not, why might a more complex model show better out-of-sample performance?

Minimum OOS MSPE does occur at degree 2. We quickly start to diverge as model complexity increases.

Sample Size and Model Complexity

The bias-variance tradeoff depends on sample size. With more data, we can support more complex models without overfitting.

```
## Function to compare models across sample sizes
compare_models_by_N <- function(
  N_values,
  formula_simple,
  formula_complex
) {
  results <- data.frame(
    N = numeric(),
    model = character(),
    mspe_type = character(),
    mspe = numeric()
  )

  for (N in N_values) {
    ## Simple model
    simple_in <- evaluate_model(
      formula_simple,
      N = N,
      in_sample = TRUE
    )
```

```

simple_out <- evaluate_model(
  formula_simple,
  N = N,
  in_sample = FALSE
)

## Complex model
complex_in <- evaluate_model(
  formula_complex,
  N = N,
  in_sample = TRUE
)
complex_out <- evaluate_model(
  formula_complex,
  N = N,
  in_sample = FALSE
)

results <- rbind(
  results,
  data.frame(
    N = rep(N, 4),
    model = c("Simple", "Simple", "Complex", "Complex"),
    mspe_type = c(
      "In-sample",
      "Out-of-sample",
      "In-sample",
      "Out-of-sample"
    ),
    mspe = c(
      mean(simple_in),
      mean(simple_out),
      mean(complex_in),
      mean(complex_out)
    )
  )
)
}

return(results)
}

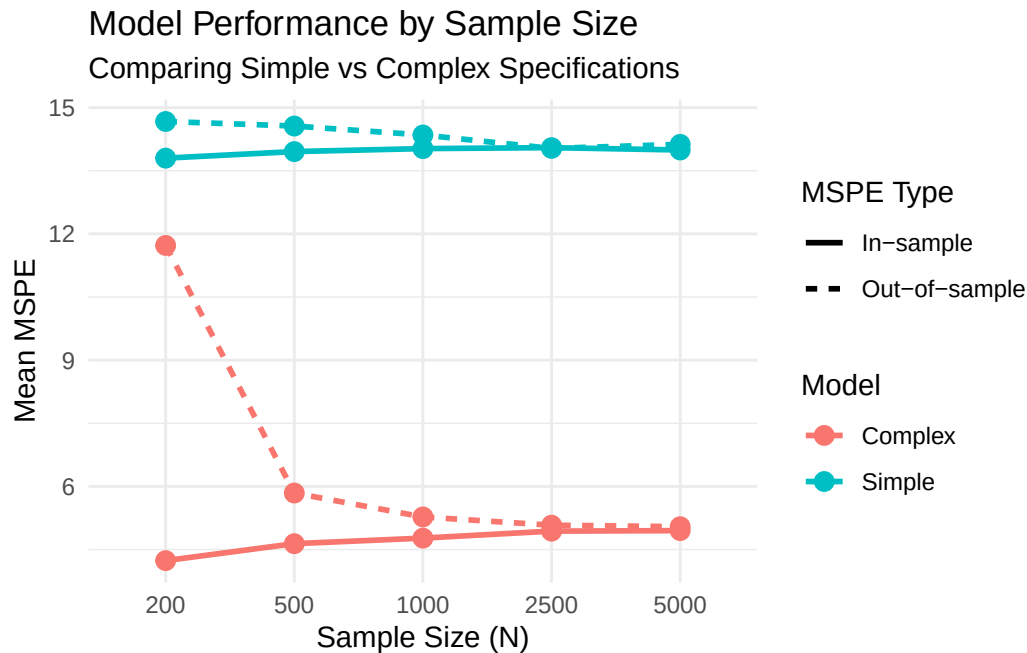
```

```

## Run comparison (using small B for speed)
N_values <- c(100, 200, 500, 1000, 2500, 5000)
set.seed(20260210)
comparison_results <- compare_models_by_N(
  N_values = N_values,
  formula_simple = y ~ x1 + x2 + i(group),
  formula_complex = y ~ i(group) +
    i(group, x1) +
    i(group, x1^2) +
    i(group, x1^3) +
    i(group, x1^4) +
    i(group, x2) +
    i(group, x2^2) +
    i(group, x2^3) +
    i(group, x2^4)
)

## Plot results
ggplot(
  comparison_results |> filter(N >= 200),
  aes(x = factor(N), y = mspe, color = model, linetype = mspe_type)
) +
  geom_line(aes(group = interaction(model, mspe_type)), linewidth = 1) +
  geom_point(size = 3) +
  labs(
    title = "Model Performance by Sample Size",
    subtitle = "Comparing Simple vs Complex Specifications",
    x = "Sample Size (N)",
    y = "Mean MSPE",
    color = "Model",
    linetype = "MSPE Type"
  ) +
  theme_minimal()

```



Exercise

1. How does the gap between in-sample and out-of-sample performance change as sample size increases for the complex model? What does this tell us about the bias-variance tradeoff?

A: As sample size increases, we see the gap between in-sample and out-of-sample performance decreasing. With a smaller sample size, we have high variance where each data point has a stronger effect on the shape of the curve. As sample size increases, we reduce variance, and new points aren't wildly altering the shape of the line.

Complex models often have lower bias (fit the true model) better than simpler models, but run more in danger of over-fitting.

Key Takeaways

1. **Model Specification Matters:** When the true functional form is unknown, different specifications can lead to very different prediction performance.
2. **In-Sample vs Out-of-Sample:** In-sample fit can be misleading due to overfitting. Out-of-sample evaluation provides a more honest assessment of how well the model will perform on new data.

3. **Bias-Variance Tradeoff:** Simple models may underfit (high bias), while complex models may overfit (high variance). The optimal model balances these concerns.
4. **Sample Size:** With larger samples, we can afford more complex models without overfitting. With small samples, simpler models often generalize better.